BAB I
PENDAHULUAN

1.1 Latar Belakang

Manajemen tugas merupakan aspek yang sangat penting dalam keberhasilan proyek kolaboratif yang seharusnya dapat dikelola secara terorganisir, transparan, dan *real-time*. Dalam kondisi yang diharapkan, setiap anggota tim dapat memantau perkembangan kerja, mengidentifikasi hambatan sejak dini, serta memiliki pembagian tanggung jawab yang jelas melalui dukungan teknologi manajemen proyek. Dengan adanya sistem visualisasi alur kerja yang optimal, diharapkan efisiensi kerja tim dapat meningkat dan risiko keterlambatan proyek dapat diminimalisir [1] .

Namun pada kenyataannya, banyak mahasiswa dan tim kecil masih menghadapi kendala dalam koordinasi tugas. Penggunaan platform komunikasi instan atau daftar tugas (*to-do list*) statis sering kali menyebabkan ketidakteraturan data dan hilangnya fokus akibat fenomena *cognitive overload* [2]. Meskipun sistem manajemen proyek populer seperti Trello atau Notion telah banyak diadopsi, mayoritas sistem tersebut masih memiliki batasan dalam akses kontrol pada versi gratis atau justru memiliki kompleksitas fitur yang terlalu tinggi bagi pengguna pemula. Hal ini diperparah oleh koordinasi tim dinamis yang membutuhkan pemantauan aktivitas secara mendetail dan pembagian peran yang spesifik guna menghindari tumpang tindih tanggung jawab.

Beberapa platform telah memanfaatkan metode Kanban untuk visualisasi tugas [1]. Meskipun demikian, sering kali implementasinya belum sepenuhnya mempertimbangkan aspek keamanan hak akses (RBAC) pada skala tim kecil tanpa biaya tambahan [3]. Selain itu, performa manajemen alur kerja sangat bergantung pada kesederhanaan antarmuka agar tidak mengganggu fokus pengguna pada tugas utama [4]. Berdasarkan kesenjangan tersebut, diperlukan suatu solusi sistem

manajemen tugas yang tidak hanya mampu melakukan visualisasi alur kerja secara optimal, tetapi juga menyediakan ruang kerja kolaboratif yang aman dan minimalis.

Berdasarkan kesenjangan tersebut, diperlukan suatu solusi sistem manajemen tugas yang tidak hanya mampu melakukan visualisasi alur kerja secara optimal, tetapi juga menyediakan ruang kerja kolaboratif yang aman dan minimalis. Oleh karena itu, pada laporan ini diusulkan perancangan dan implementasi Decko, sebuah sistem manajemen tugas berbasis web yang menerapkan Metode Kanban untuk optimalisasi alur kerja. Sistem ini dikembangkan menggunakan *tech stack* modern yang terdiri dari React + Vite di sisi *frontend*, Express di sisi *backend*, serta basis data relasional PostgreSQL. Sistem ini juga mengintegrasikan fitur *Group Workspace* dengan sistem *Role-Based Access Control* (RBAC) 4 tingkatan (*Owner*, *Admin*, *Member*, *Observer*) serta komunikasi *real-time* berbasis Socket.io untuk menjamin integritas serta keamanan data kolaborasi tim secara efektif.

1.2 Deskripsi Aplikasi

Terinspirasi langsung dari Trello dan Notion, Decko merupakan sebuah aplikasi manajemen proyek atau tugas berbasis web yang dirancang untuk mempermudah individu maupun tim dalam mengelola alur kerja. Nama "Decko" diambil dari kata “Deck” yang memiliki arti tumpukan kartu yang menjadi simbol utama dalam penyusunan tugas. Aplikasi ini tidak sekadar berfungsi sebagai media pencatatan, tetapi bertindak sebagai pusat kendali visual yang menerapkan Metode Kanban.

Melalui metode ini, pengguna dapat membagi proyek besar menjadi kepingan tugas kecil berbentuk kartu (*Card*) yang kemudian dipindahkan secara bertahap melalui kolom-kolom status (*List*), seperti dari tahap persiapan menuju pengerjaan hingga selesai. Hal ini memberikan gambaran yang sangat jelas mengenai posisi setiap tugas dalam sebuah garis waktu proyek. Selain itu, Decko mengintegrasikan fitur *Group Workspace* yang memungkinkan banyak pengguna untuk berkolaborasi dalam satu papan yang sama secara *real-time* melalui protokol Socket.io.

Untuk menjaga ketertiban data di dalam kolaborasi tim, aplikasi ini dilengkapi dengan sistem pengaturan peran yang disebut *Role-Based Access Control* (RBAC). Sistem ini memungkinkan pemilik proyek untuk menentukan tingkat akses bagi setiap anggota melalui empat tingkatan peran, yaitu *Owner*, *Admin*, *Member*, dan *Observer*. Dari sisi tampilan, Decko mengusung konsep *Minimalist Design* dengan penerapan tema gelap dan aksen biru yang berfokus pada fungsi inti, sehingga pengguna tidak akan merasa bingung dengan banyaknya tombol atau fitur yang tidak perlu. Dengan begitu, Decko menjadi solusi untuk menjaga produktivitas tetap stabil di tengah tumpukan pekerjaan yang kompleks.

1.3 Tujuan Pengembangan

Tujuan umum :

1. Menciptakan platform manajemen tugas sederhana yang bisa meningkatkan produktifitas individu maupun kelompok melalui visualisasi alur kerja yang efektif

Tujuan Khusus :

1. Mengimplementasikan metode Kanban pada sistem digital untuk memudahkan pelacakan alur kerja
2. Mengimplementasikan fitur *Group Workplace* yang memungkinkan kolaborasi antar anggota tim secara efektif dan transparan.
3. Menyediakan sistem pengaturan hak akses melalui *Role-Based Access Control* (RBAC) bertingkat (*Owner*, *Admin*, *Member*, *Observer*) untuk menjamin keamanan dan integritas data di dalam ruang kerja tim.
4. Menghasilkan antarmuka aplikasi dengan konsep premium minimalis menggunakan tema gelap guna meningkatkan efisiensi pengguna dalam mengelola pengerjaan proyek.

1.4 Batasan Masalah

Agar pengembangan aplikasi tetap terfokus, batasan masalah aplikasi Decko ditetapkan sebagai berikut:

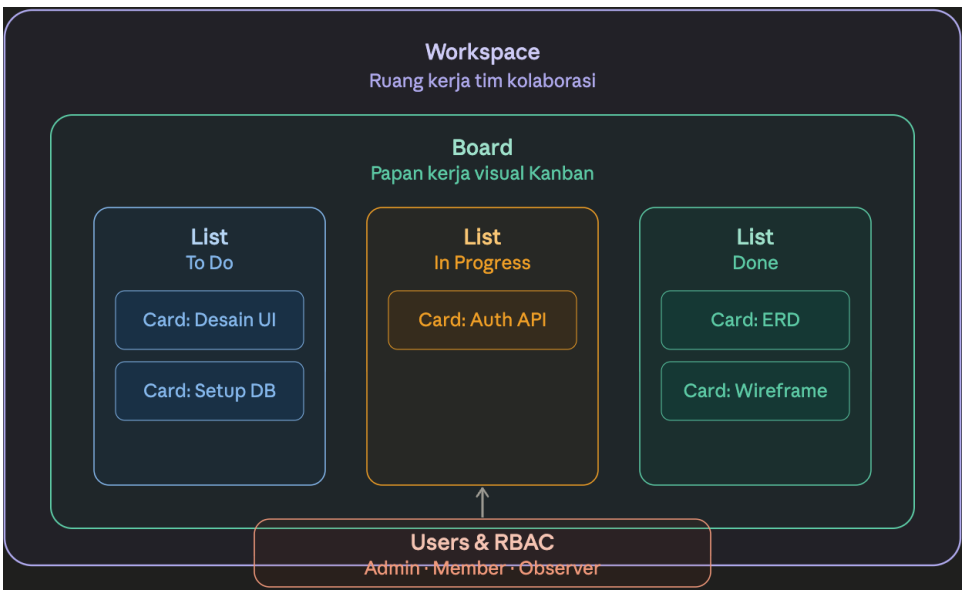
1. Aplikasi hanya dikembangkan sebagai platform berbasis web (*Web Application*) dengan menggunakan framework React dan Vite yang bisa diakses di peramban desktop.
2. Target pengguna aplikasi Decko ini dikhususkan untuk mahasiswa, tim proyek kecil, atau individu yang memerlukan aplikasi manajemen tugas sederhana.
3. Sistem manajemen tugas berfokus pada fungsionalitas inti Kanban (*Board*, *List*, *Card*, *Label*, *Priority*, *Deadline*, *Comment*) dan fitur *Group Workspace*.
4. Fitur kolaborasi dibatasi pada pengaturan peran (*Owner*, *Admin*, *Member*, *Observer*) di dalam lingkup satu ruang kerja (*Workspace*), di mana proses manajemen anggota diakses melalui komponen *Settings Modal* pada halaman *Board*.
5. Sistem komunikasi *real-time* di dalam aplikasi dibatasi pada sinkronisasi pergeseran kartu, pembaruan data Kanban, dan kolaborasi papan tugas via Socket.io, tidak mencakup fitur komunikasi *chatting* teks langsung atau panggilan video.

BAB II
DESKRIPSI SISTEM

2.1 Gambaran Umum Aplikasi

Decko merupakan aplikasi manajemen tugas berbasis web yang dirancangan untuk mengoptimalisasi alur kerja kolaboratif melalui metode visual kanban [1]. Secara teknis, aplikasi ini bekerja dengan memuat proses alur kerja ke dalam papan (*Board*) yang terdiri dari kolom status (*List*) dan kartu tugas (*Card*).

Aplikasi ini dibuat menggunakan React untuk sisi antarmuka, Express (Node.js) untuk penanganan server logika bisnis, serta PostgreSQL sebagai sistem manajemen basis data relasional. Dengan antarmuka yang mengusung prinsip premium minimalis (*dark theme*), Decko memungkinkan pengguna mengelola tugas dengan fitur *drag-and-drop* yang interaktif. Setiap perpindahan kartu maupun pembaruan data massal diproses secara efisien melalui API *batch-update* dan disinkronisasikan secara *real-time* ke pengguna lain yang terhubung menggunakan Socket.io setelah melewati validasi *middleware* kontrol akses di sisi *backend*.



Gambar 1 Gambaran Umum Aplikasi

2.2 Stakeholder dan User

Secara umum, target pengguna aplikasi Decko difokuskan pada individu, mahasiswa, dan tim proyek skala kecil yang membutuhkan alat bantu manajemen kerja yang praktis. Namun, di dalam sistem aplikasi, tingkatan pengguna (*user level*) diatur berdasarkan hak aksesnya menggunakan metode *Role-Based Access Control* (RBAC) pada setiap ruang kerja (*Workspace*) [5].

Hal ini berarti setiap pendaftar akun memiliki kedudukan yang sama di awal, namun perannya akan ditentukan secara spesifik di dalam setiap papan proyek yang diikuti. Hak akses ke dalam suatu ruang kerja diperoleh melalui pemanfaatan kode undangan (*invitation code*) unik. Pembagian empat peran tersebut dijelaskan sebagai berikut:. Pembagian peran tersebut dijelaskan sebagai berikut:

2.2.1 Owner

1. Karakteristik: Pengguna yang pertama kali menginisiasi atau membuat ruang kerja (*Workspace*) baru secara otomatis bertindak sebagai *Owner* di dalam *Workspace* tersebut. Peran ini berlaku mutlak dan memegang kepemilikan tertinggi atas ruang kerja yang bersangkutan.
2. Hak Akses: Memiliki kendali penuh (*full control*) untuk membuat, menyunting pengaturan, maupun menghapus *Workspace*, membuat kode undangan baru, mengubah hak akses anggota menjadi Admin/Member/Observer, serta memiliki hak eksklusif untuk menghapus seluruh papan proyek (*Board*) yang ada di dalam ruang kerja tersebut.

2.2.2 Admin

1. Karakteristik: Pengguna yang dipercayai oleh *Owner* untuk membantu mengelola operasional ruang kerja dan kolaborasi tim.
2. Hak Akses: Memiliki kendali yang hampir setara dengan *Owner*, seperti membuat atau menghapus *Board*, memanipulasi *List* dan *Card*, serta mengelola keanggotaan tim melalui *Settings Modal* (mengubah peran *Member* atau *Observer*). Namun, *Admin* memiliki batasan tidak dapat

menghapus ruang kerja (*Workspace*) dan tidak dapat mengubah atau mengeluarkan status *Owner* dan *Admin* lain

2.2.3 Member (Editor)

1. Karakteristik: Pengguna yang bergabung ke dalam ruang kerja melalui *invitation code* dan diberi hak oleh *Owner* atau *Admin* untuk menjadi kontributor aktif proyek.
2. Hak Akses: Memiliki izin penuh dalam aspek operasional papan tugas, seperti membuat kartu tugas (*Card*), menyunting deskripsi, memberikan komentar (*Comments*), memberikan label (*Labels*), mengatur prioritas (*Priority*), menetapkan tenggat waktu (*Deadline*), mengalokasikan penugasan diri atau tim lain (*Card Members*), serta memindahkan kartu antarkolom status (*List*). Namun, *Member* tidak memiliki hak untuk mengakses *Settings Modal* manajemen anggota ataupun mengubah konfigurasi utama papan kerja.

2.2.4 Observer (Viewer)

1. Karakteristik: Pihak luar atau peninjau yang diundang ke dalam ruang kerja hanya untuk memantau perkembangan proyek tanpa terlibat langsung dalam pengisian tugas.
2. Hak Akses: Hanya memiliki izin untuk melihat visualisasi papan Kanban, membaca detail kartu tugas, melihat label, tingkatan prioritas, tenggat waktu, serta membaca komentar tanpa hak akses untuk menambah, mengubah, mengomentari, atau menggeser data apa pun (*Read-Only*)

2.3 Kebutuhan Fungsional

Kebutuhan fungsional mendefinisikan fitur utama yang harus ada agar aplikasi berfungsi sebagai alat kolaborasi yang kokoh:

1. Manajemen Autentikasi: Sistem harus mampu menangani pendaftaran akun baru (menggunakan *email*, *username*, dan *password*) serta proses masuk (*login*) menggunakan kredensial *username* dan *password* secara aman.
2. Manajemen Workspace & Board via Kode Undangan: Sistem harus mendukung pembuatan ruang kerja (*Workspace*) dan papan proyek (*Board*), serta menyediakan fitur pembuatan dan validasi kode undangan (*invitation code*) agar pengguna lain dapat bergabung.
3. Operasi CRUD & Batch-Update Kanban: Sistem harus mampu menjalankan fungsi membuat, membaca, memperbarui, dan menghapus pada entitas *List*, *Card*, *Labels*, *Comments*, *Priority*, dan *Deadline*. Sistem juga harus mendukung pembaruan posisi massal (*batch-update*) saat terjadi reorganisasi urutan elemen.
4. Otorisasi Role-Based Access Control (RBAC): Sistem harus secara otomatis memvalidasi tingkatan peran pengguna (*Owner*, *Admin*, *Member*, *Observer*) di sisi *backend* melalui *middleware* setiap kali ada permintaan manipulasi data ruang kerja.
5. Sinkronisasi Real-Time: Sistem harus mampu menyebarkan data perubahan papan tugas (seperti aksi *drag-and-drop* kartu atau perubahan isi kartu) secara instan kepada seluruh pengguna yang sedang aktif di papan yang sama memanfaatkan teknologi Socket.io [6].

2.4 Kebutuhan Non-Fungsional

Kebutuhan non-fungsional menjelaskan standar kualitas dan batasan yang harus dipenuhi agar aplikasi tidak hanya berjalan secara teknis, tetapi juga memberikan pengalaman penggunaan yang baik dan aman:

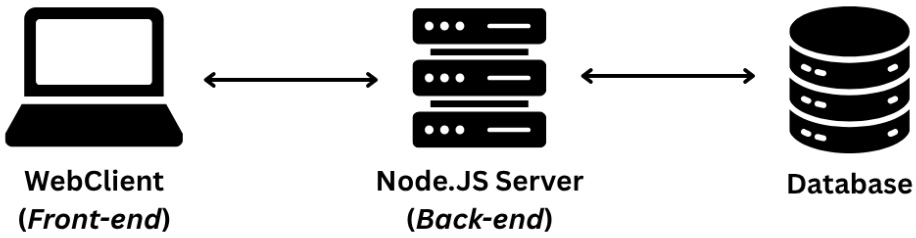
1. Keamanan: Implementasi keamanan menggunakan *Middleware* otorisasi dan JWT (*JSON Web Token*) yang disimpan secara aman untuk melindungi setiap rute API, sehingga menjamin hanya pengguna dengan identitas valid yang dapat berinteraksi dengan basis data PostgreSQL.
2. Kemudahan Penggunaan (Aestetika Premium): Desain antarmuka yang mengadopsi tema gelap yang elegan guna mengurangi visual *clutter* dan *cognitive overload*, memastikan kenyamanan mata pengguna saat memantau visualisasi papan Kanban dalam durasi yang lama.
3. Integritas & Konsistensi Data: Arsitektur database PostgreSQL diatur dengan relasi tabel terstruktur (termasuk tabel *many-to-many* untuk alokasi tugas dan label) guna menjaga konsistensi data dari anomali data saat diakses bersamaan (*concurrent access*).
4. Performa dan Latensi Rendah: Waktu respon sistem dalam memproses pergeseran kartu harus di bawah ambang batas kritis melalui optimasi kueri basis data dan API *batch-update*, didukung transfer data *real-time* Socket.io yang efisien.

BAB III
PERANCANGAN SISTEM

3.1 Arsitektur Sistem

Arsitektur sistem Decko dirancang menggunakan model *Client-Server* berbasis *Three-Tier Architecture* yang memisahkan antara antarmuka pengguna, logika bisnis, dan penyimpanan data [7]. Komponen utama arsitektur ini meliputi:

1. Frontend (Client Tier): Dibangun menggunakan React + Vite. Berfungsi untuk merender papan Kanban secara visual dengan performa rendering yang cepat, menangani interaksi *drag-and-drop* kartu tugas menggunakan pustaka stateful, mengelola koneksi *client-side* Socket.io, serta mengirimkan permintaan (*request*) berupa token JWT ke sisi *backend*.
2. Backend (Application Tier): Dibangun menggunakan framework Express (Node.js). Sisi *server* ini bertanggung jawab memproses seluruh logika bisnis aplikasi, menyediakan RESTful API, mengelola *lifecycle* koneksi WebSocket via Socket.io untuk kebutuhan *real-time notification*, serta menyediakan *Middleware* Keamanan untuk melakukan verifikasi peran pengguna (*Owner*, *Admin*, *Member*, *Observer*) sebelum mengizinkan manipulasi data.
3. Database (Data Tier): Menggunakan PostgreSQL sebagai *Relational Database Management System* (RDBMS). Lapisan ini menyimpan data pengguna secara terstruktur, relasi keanggotaan ruang kerja, serta seluruh entitas manajemen tugas beserta atribut-atribut pelengkapnya seperti komentar, prioritas, tenggat waktu, dan sistem pewarnaan label.



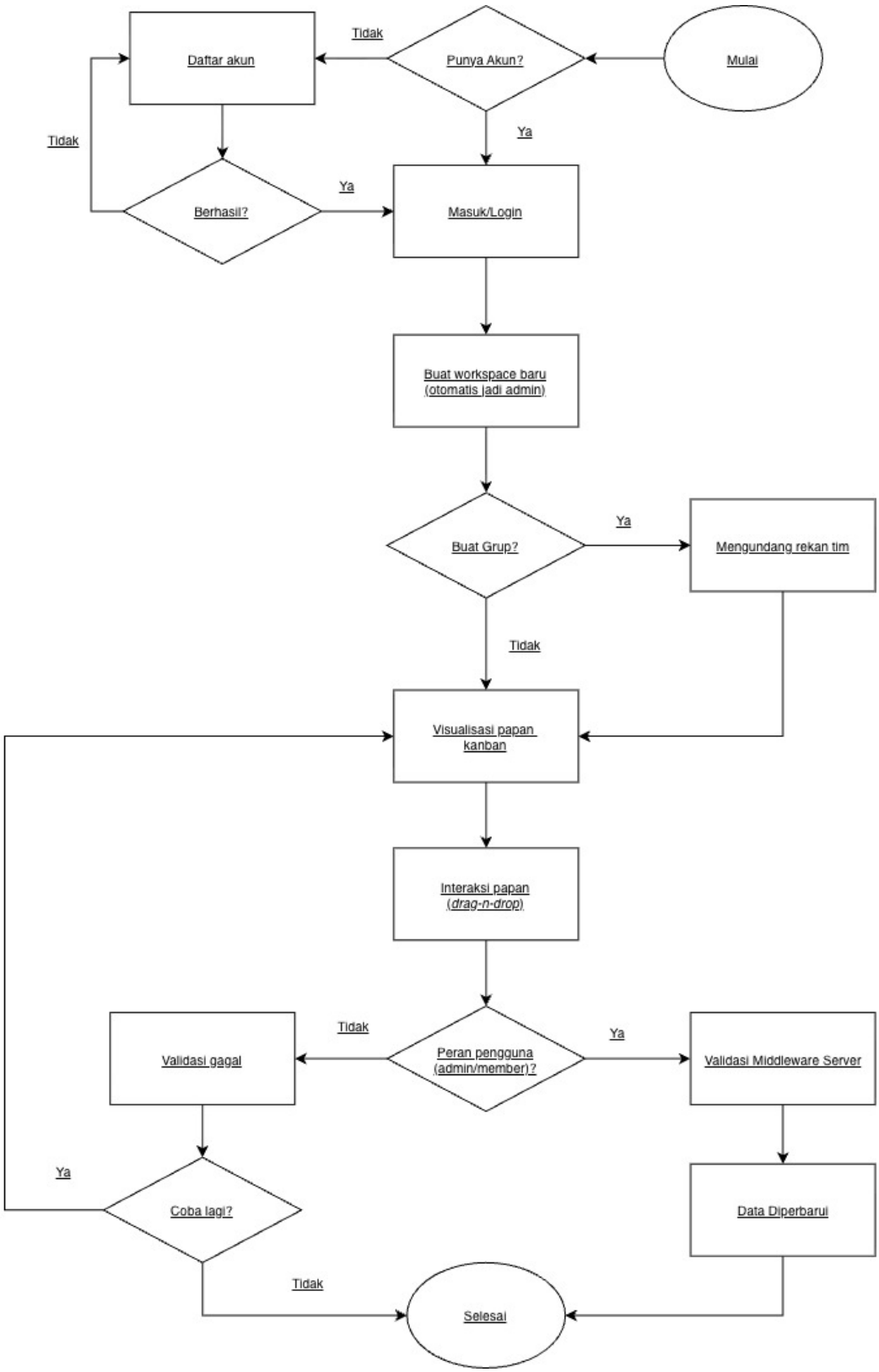
Gambar 2 Arsitektur Sistem

3.2 Workflow Sistem

Sistem alur kerja (*workflow*) operasional aplikasi Decko dibagi berdasarkan tingkat otorisasi peran pengguna (RBAC) dan integrasi *real-time* untuk memastikan integritas dan keamanan data saat berkolaborasi di dalam papan Kanban. Penjelasan alur interaksi tersebut dijabarkan sebagai berikut:

1. Tahap Autentikasi dan Registrasi: Pengguna mendaftarkan akun baru dengan memasukkan *email*, *username*, dan *password*. Setelah akun terdaftar, pengguna masuk menggunakan *username* dan *password*, di mana sistem akan memberikan token JWT sebagai tanda sesi aktif.
2. Inisialisasi & Kolaborasi Ruang Kerja: Ketika pengguna membuat sebuah *Workspace* baru, sistem secara otomatis menetapkannya sebagai *Owner*. Untuk berkolaborasi, *Owner* dapat menyalin kode undangan (*invitation code*) yang diatur di dalam sistem untuk disebarkan kepada calon anggota. Calon anggota memasukkan kode tersebut untuk terdaftar ke dalam tabel keanggotaan ruang kerja.
3. Tahap Operasional Papan Kanban (Core Workflow): Saat pengguna berada di dalam halaman *Board* Kanban dan melakukan aksi manipulasi data (seperti menambah kartu, mengubah deskripsi, atau melakukan aksi *drag-and-drop* kartu tugas untuk merestrukturisasi kolom), komponen React akan mengirimkan permintaan *batch-update* posisi atau payload perubahan ke server Express beserta token JWT.

4. Logika Keamanan Middleware: *Middleware* di server Express mengecek validitas token JWT dan membaca tingkatan peran pengguna pada tabel keanggotaan *Workspace*
    - Jika peran pengguna adalah *Observer*, *middleware* akan menolak *request*, membatalkan perpindahan kartu secara otomatis di sisi klien, dan menampilkan notifikasi penolakan akses.
    - Jika peran pengguna adalah *Owner*, *Admin*, atau *Member*, *middleware* meloloskan verifikasi, lalu server Express akan mengeksekusi kueri pembaruan ke database PostgreSQL.



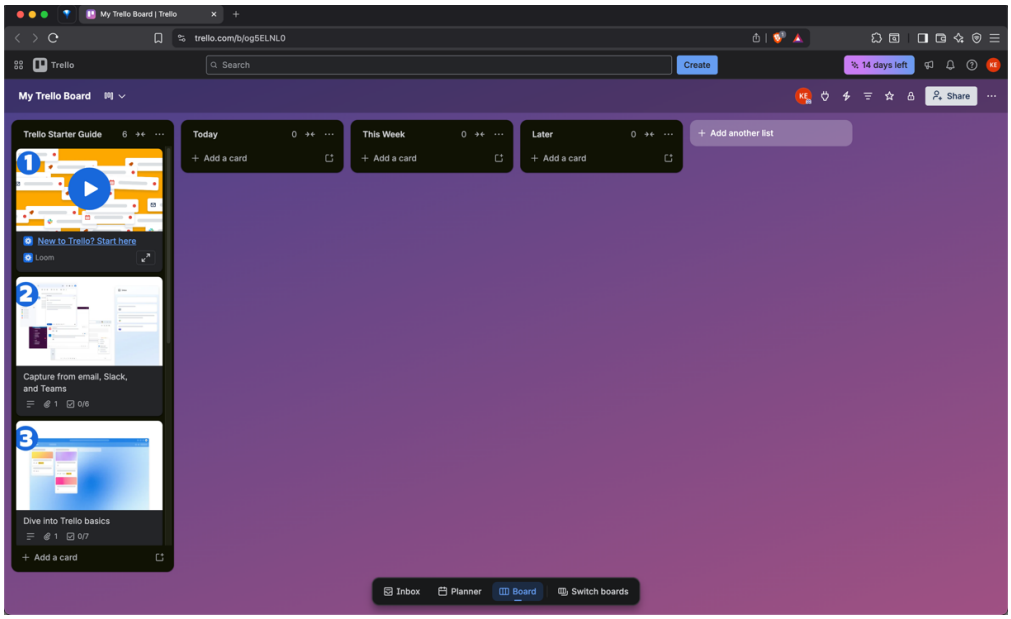
Gambar 3 *Workflow* Decko

3.3 Class Diagram

Sebelum memetakan arsitektur penyimpanan ke dalam *Class Diagram*, dilakukan tahapan analisis kebutuhan data untuk menstrukturkan entitas, menentukan atribut, serta menetapkan aturan bisnis yang berlaku di dalam aplikasi Decko. Tahapan ini penting untuk memastikan seluruh komponen data memiliki asal-usul kebutuhan yang jelas, terarah, serta tervalidasi oleh standar arsitektur industri.

3.3.1 Analisis Komparatif Aplikasi Serupa

Sebagai bukti otentik dasar pembentukan entitas basis data kolaboratif, dilakukan analisis komparatif dengan mengacu pada struktur data Trello, salah satu platform manajemen tugas berbasis Kanban yang paling banyak diadopsi secara global. Berdasarkan arsitektur data Trello, fondasi utama manajemen tugas bertumpu pada konsep *Workspace-scoped models*.



Gambar 4 Trello

Berikut adalah matriks adopsi struktur data yang menjadi landasan otentik pengembangan entitas pada aplikasi Decko yang telah diselaraskan dengan basis data PostgreSQL:

Tabel 1 Komparasi Trello dan Decko

Struktur Komponen Trello	**Implementasi pada Decko**	**Validasi dan Aturan Bisnis**
Organizations / Workspaces	workspaces	Menjadi wadah utama isolasi data antar tim kolaborasi agar data antar organisasi tidak bercampur.
Members & Memberships	workspace_members	Mengadopsi tabel perantara khusus untuk mengunci peran (*Role*) pengguna di tingkat ruang kerja, membagi otorisasi menjadi *Owner*, *Admin*, *Member*, dan *Observer*.
Boards	boards	Papan kerja visual yang terikat secara mutlak ke dalam satu *workspace* induk.
Lists	lists	Kolom status yang menggunakan parameter indeks posisi untuk melacak urutan alur kerja dari kiri ke kanan secara dinamis.
Cards	cards	Unit tugas terkecil yang memiliki parameter relasi fleksibel ke entitas list untuk mendukung perubahan data instan saat terjadi aksi *drag-and-drop*.

3.3.2 Identifikasi Entitas dan Atribut

Penentuan atribut spesifik pada setiap entitas di dalam database PostgreSQL Decko didasarkan pada hasil komparasi industri, kebutuhan fungsional sistem kolaboratif, serta optimalisasi relasi *many-to-many*. Berikut adalah kamus data (*metadata*) yang menjadi landasan pembentukan basis data Decko:

1. Entitas users

   Asal-usul Kebutuhan: Diperlukan untuk menyimpan informasi akun pengguna yang telah melewati proses registrasi dan autentikasi agar dapat masuk ke dalam sistem. Metadata Atribut:
   - id (SERIAL): Kunci utama (Primary Key) sebagai identifikasi unik setiap pendaftar sistem.
   - username (VARCHAR 50): Menyimpan nama pengguna unik yang digunakan sebagai kredensial log masuk (login).
   - password_hash (TEXT): Menyimpan kata sandi akun yang telah dienkripsi menggunakan metode hashing demi keamanan data.
   - email (VARCHAR 100): Menyimpan alamat surel unik pengguna.
2. Entitas workspace_members

   Asal-usul Kebutuhan: Memetakan hubungan banyak ke banyak (*many-to-many*) antara pengguna dan ruang kerja, sekaligus menyimpan parameter tingkat otorisasi keamanan (RBAC) dalam lingkup *Workspace.* Metadata Atribut:
   - workspace_id (INT): Kunci tamu (Foreign Key) yang merujuk pada entitas workspaces.
   - user_id (INT): Kunci tamu (Foreign Key) yang merujuk pada entitas users.
   - role (VARCHAR 20): Menyimpan batasan peran penentu hak akses ('Owner', 'Admin', 'Member', 'Observer').
   - joined_at (TIMESTAMP WITH TIME ZONE): Menyimpan tanggal dan waktu ketika anggota bergabung ke dalam workspace.

   Composite Primary Key dibentuk dari kombinasi (workspace_id, user_id).

3. Entitas workspaces

   Asal-usul Kebutuhan: Wadah utama untuk mengelompokkan satu atau beberapa papan proyek yang digunakan oleh individu maupun tim kolaboratif. Metadata Atribut:

   - id (SERIAL): Kunci utama (Primary Key) unik dari setiap ruang kerja yang tercipta.
   - name (VARCHAR 100): Penamaan ruang kerja untuk mempermudah identifikasi manajemen tim.
   - invitation_code (VARCHAR 10): Menyimpan string unik acak yang berfungsi sebagai kode akses masuk bagi pengguna lain yang ingin bergabung ke dalam *Workspace*.
   - owner_id (INT): Pencatatan identitas pengguna yang bertindak sebagai Owner di dalam Workspace (merujuk ke id pada users).

4. Entitas boards

   Asal-usul Kebutuhan: Papan kerja visual utama yang mengadopsi metode Kanban untuk melacak status pengerjaan sebuah proyek di dalam *workspace* tertentu. Metadata Atribut:

   - id (SERIAL): Kunci utama (Primary Key) unik untuk setiap papan Kanban.
   - name (VARCHAR 100): Nama atau judul proyek pada papan Kanban.
   - workspace_id (INT): Kunci tamu (Foreign Key) untuk mengikat papan agar berada di dalam ruang kerja yang tepat.

5. Entitas lists

   Asal-usul Kebutuhan: Representasi kolom status alur kerja yang membagi tahapan kartu tugas di dalam papan. Metadata Atribut:

   - id (SERIAL): Kunci utama (Primary Key) dari kolom status.
   - name (VARCHAR 100): Judul kolom status alur kerja.
   - order_index (INT): Atribut penentu urutan visual kolom dari kiri ke kanan secara dinamis.

- board_id (INT): Kunci tamu (Foreign Key) yang merujuk pada entitas boards.

6. Entitas cards

   Asal-usul Kebutuhan: Unit tugas terkecil yang merepresentasikan kepingan pekerjaan spesifik yang diisi dan digeser oleh tim kolaborasi. Metadata Atribut:

   - id (SERIAL): Kunci utama (Primary Key) unik dari kartu tugas digital.
   - title (VARCHAR 200): Judul atau nama ringkas dari tugas kerja.
   - description (TEXT): Catatan atau detail instruksi pekerjaan di dalam kartu.
   - order_index (INT): Atribut penentu urutan kartu secara vertikal (atas ke bawah) dalam kolom status.
   - priority (VARCHAR 20): Tingkat kepentingan pengerjaan tugas (misalnya: Low, Medium, High).
   - deadline (TIMESTAMP WITH TIME ZONE): Menyimpan informasi batas waktu atau tenggat pengerjaan tugas.
   - list_id (INT): Kunci tamu (Foreign Key) penunjuk lokasi kolom, yang nilainya akan terus diperbarui secara dinamis saat terjadi aksi interaksi drag-and-drop.
   - created_by (INT): Kunci tamu (Foreign Key) yang merujuk pada pembuat kartu (users).

7. Entitas card_members (Intersection)

   Asal-usul Kebutuhan: Mengakomodasi penugasan satu atau lebih anggota tim (*assignee*) ke dalam suatu kartu tugas secara fleksibel. Metadata Atribut:

   - card_id (INT): Kunci tamu (Foreign Key) merujuk ke entitas cards.
   - user_id (INT): Kunci tamu (Foreign Key) merujuk ke entitas users.
   - Composite Primary Key dibentuk dari kombinasi (card_id, user_id).

8. Entitas labels

   Asal-usul Kebutuhan: Menyimpan master data kategori warna atau tag yang dapat disematkan pada kartu tugas untuk penandaan visual. Metadata Atribut:

- id (SERIAL): Kunci utama (Primary Key) dari label.
- name (VARCHAR 50): Nama atau deskripsi penanda label.
- color (VARCHAR 20): Menyimpan kode warna heksadesimal penanda visual label.
- workspace_id (INT): Kunci tamu (Foreign Key) merujuk pada entitas workspaces.

9. Entitas card_labels (Intersection)

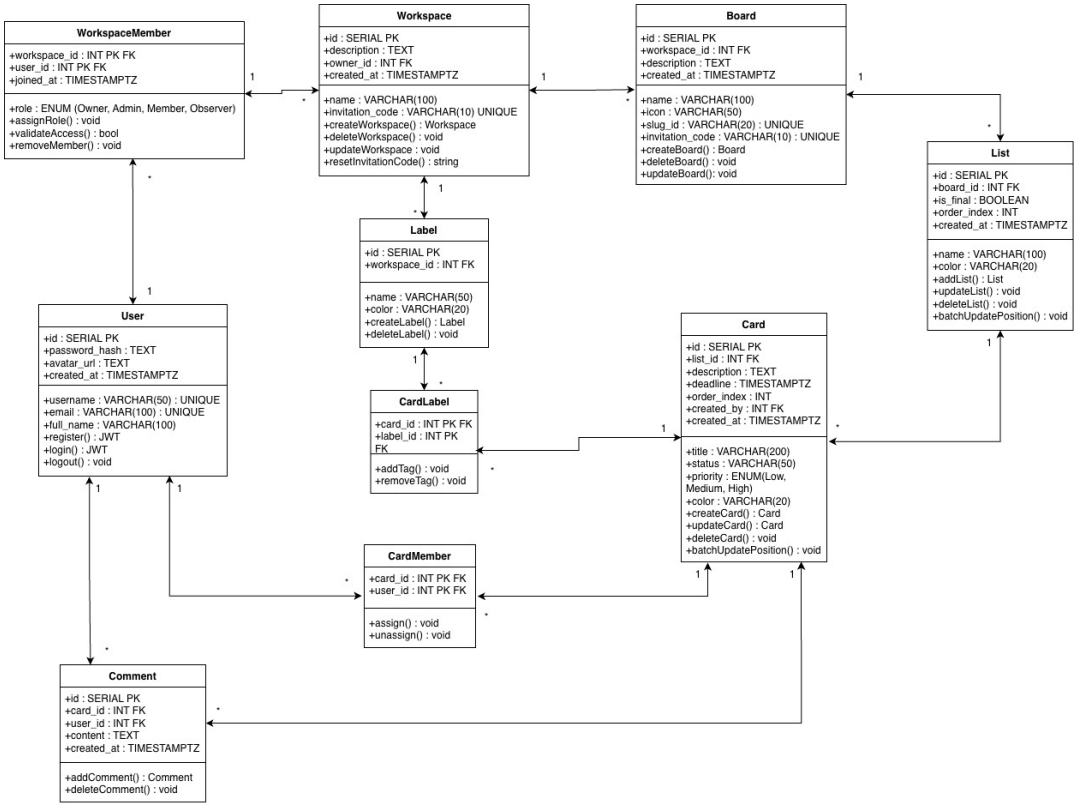
   Asal-usul Kebutuhan: Menghubungkan entitas kartu dengan berbagai label penanda visual (*many-to-many relation*). Metadata Atribut:
   - card_id (INT): Kunci tamu (Foreign Key) merujuk ke entitas cards.
   - label_id (INT): Kunci tamu (Foreign Key) merujuk ke entitas labels.
   - Composite Primary Key dibentuk dari kombinasi (card_id, label_id).

10. Entitas comments

    Asal-usul Kebutuhan: Menyediakan fitur interaksi diskusi tim berupa komentar tertulis di dalam setiap modal detail kartu tugas. Metadata Atribut:
    - id (SERIAL): Kunci utama (Primary Key) komentar.
    - content (TEXT): Isi teks komentar yang ditulis oleh pengguna.
    - created_at (TIMESTAMP WITH TIME ZONE): Waktu pembuatan komentar.
    - card_id (INT): Kunci tamu (Foreign Key) merujuk pada entitas cards.
    - user_id (INT): Kunci tamu (Foreign Key) merujuk pada entitas users (penulis komentar).

3.3.3 Visualisasi Class Diagram

Class Diagram sistem Decko dirancang untuk memodelkan struktur data objek dan logika bisnis yang mendukung arsitektur manajemen tugas kolaboratif berbasis *Role-Based Access Control* (RBAC). Diagram ini secara terperinci memetakan properti data beserta fungsi/operasi manipulasi data (*method*) di sisi server untuk menangani RESTful API, otorisasi *middleware*, hingga penyiaran data *real-time* menggunakan Socket.io.



Gambar 5 Class Diagram Decko

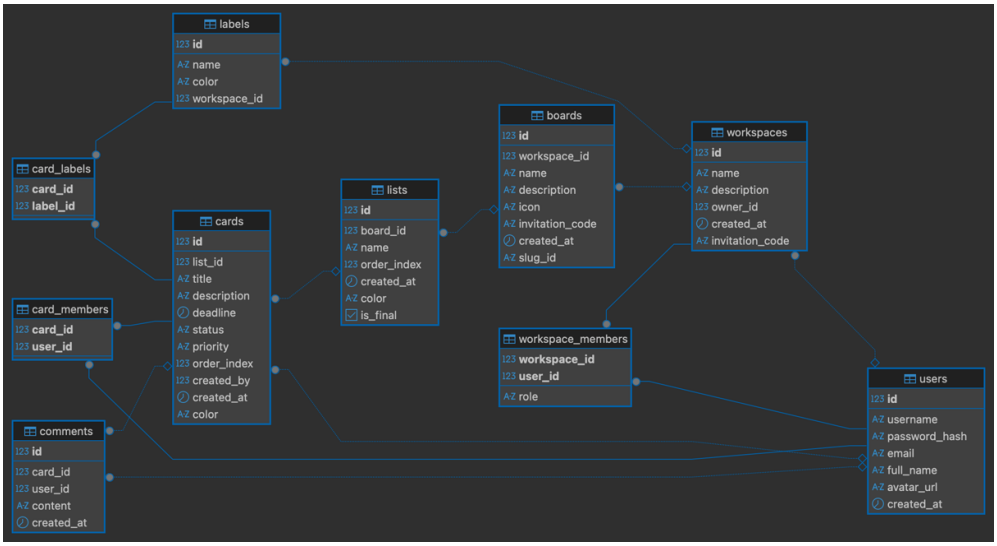
Di dalam implementasinya, kelas User bertanggung jawab penuh atas penanganan autentikasi melalui fungsi register() dan login() untuk menghasilkan token akses. Untuk menjamin keamanan hak akses berbasis *Role-Based Access Control* (RBAC), setiap permintaan manipulasi pada kelas Workspace atau Board akan melewati fungsi validateRole() pada kelas WorkspaceMember terlebih dahulu guna memastikan bahwa pengguna tidak memiliki peran *Observer* sebelum diizinkan mengeksekusi metode perubahan data.

Pada komponen inti Kanban, manajemen struktur alur kerja digerakkan oleh kolaborasi kelas List dan Card. Ketika pengguna memindahkan kolom atau menggeser kartu melalui interaksi *drag-and-drop*, sistem tidak memperbarui data satu per satu melainkan memanggil metode batchUpdatePosition() untuk mengkalkulasi ulang susunan array koordinat dan memperbarui nilai atribut order_index (INT) secara massal di database PostgreSQL.

Untuk melengkapi fungsionalitas penugasan tim dan kategorisasi data, kelas Card menyediakan metode internal untuk memanipulasi entitas perantara banyak-ke-banyak (*many-to-many*). Operasi penugasan anggota tim ditangani melalui metode assign() pada objek perantara CardMember (yang merepresentasikan tabel card_members), sedangkan penyematan tag warna visual ditangani oleh fungsi addTag() pada perantara CardLabel. Selain itu, interaksi diskusi tim diakomodasi oleh metode addComment() pada kelas Comments. Setiap kali salah satu metode mutasi atau pembaruan data pada elemen papan ini sukses dieksekusi, sistem akan secara otomatis memicu fungsi *gateway broadcasting* untuk memancarkan *event update* ke seluruh klien yang aktif di papan kerja yang sama, sehingga antarmuka React di sisi pengguna lain tersinkronisasi secara *real-time* berbasis Socket.io [6].

3.4 Entity Relationship Diagram (ERD)

Setelah struktur data dan hubungan antarkelas diidentifikasi pada bagian *Class Diagram*, perancangan tersebut kemudian ditransformasikan ke dalam bentuk *Entity Relationship Diagram* (ERD). Tahapan ini bertujuan untuk memetakan cetak biru (*blueprint*) penyimpanan data pada sistem basis data relasional (RDBMS) PostgreSQL yang akan digunakan oleh aplikasi Decko. Transformasi dari kelas komponen menjadi entitas basis data ini dilengkapi dengan spesifikasi kunci utama (*Primary Key*), kunci tamu (*Foreign Key*), serta batasan kardinalitas guna menjamin integritas data saat aplikasi diimplementasikan.



Gambar 6 ERD

3.4.1 Penjelasan Relasi (ERDish)

Hubungan relasional antar entitas di dalam *Entity Relationship Diagram* (ERD) Decko diatur berdasarkan Diagram Matrix berikut:

	user	workspace member	workspace	boards	lists	cards	card_member	labels	card_labels	comments
user		terdaftar di				membuat (created by)	ditugaskan via			menulis
workspace member	merujuk ke		ditempatkan di							
workspace		memiliki		menampung				menyediakan		
boards			dibuat di dalam		memilki					
lists				menjadi bagian dari		menampung				
cards					ditempatkan di		dialokasikan ke		disematkan	memiliki
card_member	merujuk ke					merujuk ke				
labels			milik						digunakan di	
card_labels						merujuk ke		merujuk ke		
comments	ditulis oleh					merujuk ke				

Gambar 7 Diagram Matrix

Dengan begitu, spesifikasi batas kardinalitas dan aturan bisnis yang didapat adalah seperti berikut:

1. users & workspace_members (1:M)

- Setiap pengguna (users) dapat terdaftar sebagai satu atau lebih anggota tim ruang kerja (workspace_members).
- Setiap anggota ruang kerja (workspace_members) harus merujuk pada satu dan hanya satu pengguna (users).

2. workspace & workspace_members (1:M)
   - Setiap ruang kerja (workspace) harus memiliki satu atau lebih anggota tim (workspace_members).
   - Setiap anggota tim (workspace_members) harus ditempatkan pada satu dan hanya satu ruang kerja (workspace).
3. workspace & board (1:M)
   - Setiap ruang kerja (workspace) dapat menampung satu atau lebih papan proyek Kanban (board).
   - Setiap papan proyek (board) harus dibuat di dalam satu dan hanya satu ruang kerja (workspace).
4. board & list (1:M)
   - Setiap papan proyek (board) harus memiliki satu atau lebih kolom status (list).
   - Setiap kolom status (list) harus menjadi bagian dari satu dan hanya satu papan proyek (board).
5. list & card (1:M)
   - Setiap kolom status (list) dapat menampung satu atau lebih kartu tugas (card).
   - Setiap kartu tugas (card) harus ditempatkan di dalam satu dan hanya satu kolom status (list).
6. workspace & labels (1:M)
   - Setiap papan proyek (workspace) dapat menyediakan satu atau lebih pilihan tag warna visual (labels).
   - Setiap pilihan tag warna visual (labels) harus mengacu pada satu dan hanya satu papan proyek (workspace).
7. card & card_members (1:M)

- Setiap kartu tugas (card) dapat dialokasikan kepada satu atau banyak penugasan anggota tim (card_members).
- Setiap baris penugasan anggota (card_members) harus terikat pada satu dan hanya satu kartu tugas (card).

8. card & card_labels (1:M)
   - Setiap kartu tugas (card) dapat disematkan satu atau banyak tanda label (card_labels).
   - Setiap penandaan label (card_labels) harus terikat pada satu dan hanya satu kartu tugas (card).
9. card & comments (1:M)
   - Setiap kartu tugas (card) dapat memiliki satu atau banyak baris tanggapan diskusi (comments).
   - Setiap baris tanggapan diskusi (comments) harus dicatat di dalam satu dan hanya satu kartu tugas (card)

BAB IV
DESAIN ANTARMUKA

4.1 Konsep Desain

Desain antarmuka aplikasi Decko dibangun berdasarkan prinsip *Minimalist Design* yang diintegrasikan dengan estetika premium modern. Tujuan utama dari pendekatan ini adalah untuk mengurangi *cognitive overload* pengguna agar fokus pengguna tetap tertuju pada pengelolaan tugas dan alur kerja, bukan pada antarmuka itu sendiri [2]. Secara keseluruhan, terdapat empat pilar konsep desain yang menjadi landasan perancangan tampilan aplikasi Decko, yaitu:

1. Kesederhanaan Visual Premium (Premium Visual Simplicity): Antarmuka Decko menggunakan konsep tema gelap (*dark theme*) dengan palet warna latar belakang utama gelap, dipadukan dengan aksen warna biru terang sebagai penanda fokus elemen aktif serta identitas visual utama aplikasi [4]. Tipografi menggunakan hierarki yang tegas untuk membedakan judul

papan, nama kolom, teks kartu tugas, dan metadata pendukung (prioritas dan tenggat waktu).

2. Orientasi Fungsi (Function-First): Setiap halaman dirancang agar fungsionalitas inti Kanban langsung terlihat tanpa perlu navigasi berlapis. Tata letak papan Kanban menggunakan pola horizontal berbasis kolom (*List*) yang dapat digulirkan ke samping. Komponen interaksi tingkat lanjut seperti manajemen peran anggota dipusatkan ke dalam *Settings Modal* kontekstual yang dapat diakses langsung dari pojok kanan atas halaman *Board*.
3. Konsistensi Komponen (Component Consistency): Seluruh komponen antarmuka, mulai dari kartu tugas (*Card*), kolom status (*List*), hingga modal dialog detail dan pengaturan, menggunakan ukuran sudut (*border-radius*), elevasi bayangan visual, dan jarak (*spacing*) yang konsisten.
4. Umpan Balik Interaksi Real-Time (Real-Time Interaction Feedback): Sebagai aplikasi yang mengandalkan interaksi *drag-and-drop* dan komunikasi instan via Socket.io, antarmuka dirancang untuk selalu memberikan umpan balik visual yang jelas. Saat kartu dipindahkan, area tujuan akan disorot. Apabila terjadi perubahan data secara *real-time* dari pengguna lain atau terjadi penolakan akses oleh *middleware* bagi pengguna bertipe *Observer*, antarmuka akan langsung memunculkan notifikasi toas secara *real-time*.

4.2 Mockup / Wireframe

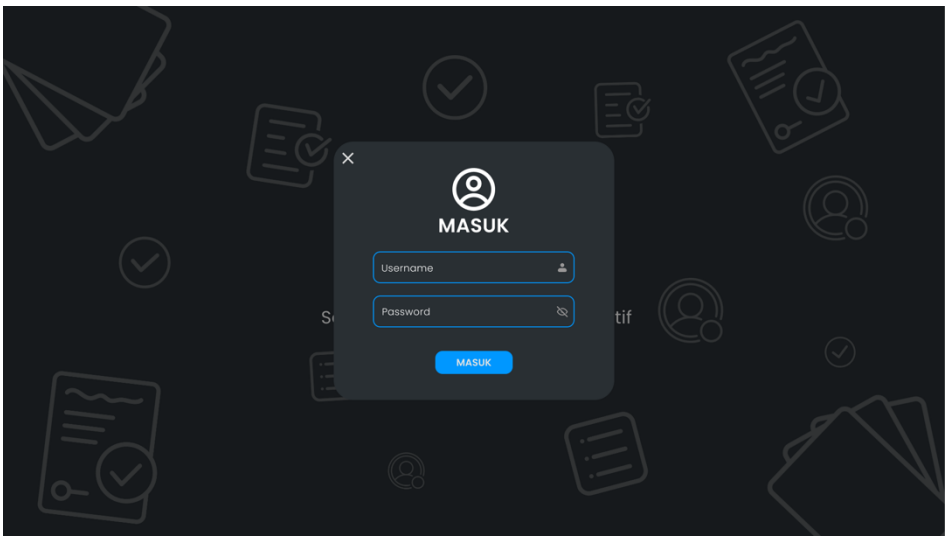
Bagian ini menyajikan rancangan tampilan awal (*wireframe*) dari setiap halaman utama aplikasi Decko. Rancangan ini mencerminkan struktur tata letak, hierarki informasi, dan posisi elemen antarmuka sebelum masuk ke tahap implementasi kode. Setiap *wireframe* dirancang berdasarkan kebutuhan fungsional dan alur kerja sistem yang telah ditetapkan

4.2.1 Halaman Autentikasi (Login & Register)

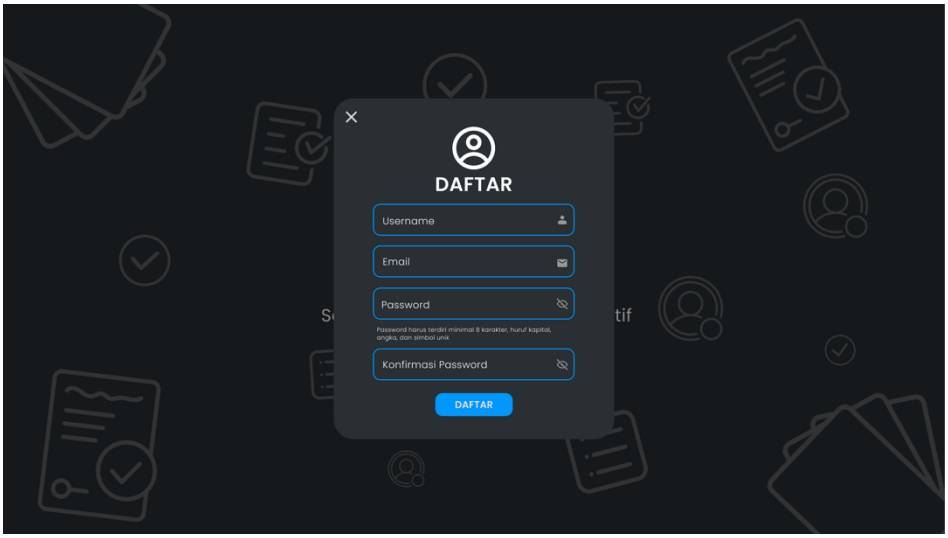
Halaman autentikasi merupakan pintu masuk utama aplikasi Decko. *Wireframe* ini menampilkan tata letak formulir yang terpusat di tengah layar dengan logo aplikasi di bagian atas, kolom input *email* dan *password*, serta tombol aksi utama. Desain ini sengaja dibuat minimalis agar pengguna dapat langsung masuk ke sistem tanpa hambatan.



Gambar 8



Gambar 9



Gambar 10

4.2.2 Halaman Dashboard (Daftar Workspace)

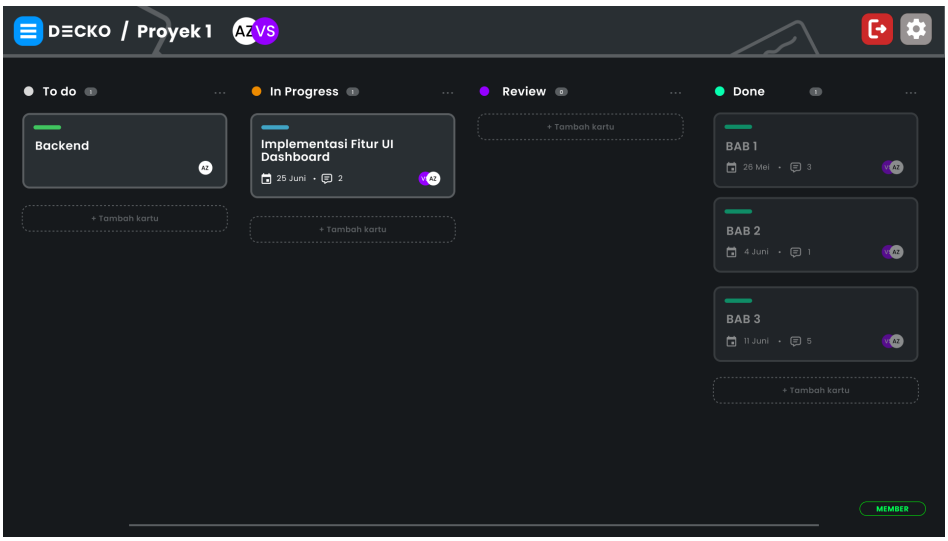
Setelah berhasil masuk, pengguna diarahkan ke halaman *Dashboard* yang menampilkan seluruh *Workspace* yang dimiliki atau diikuti oleh pengguna. Halaman ini terdiri dari *sidebar* navigasi di sisi kiri dan area konten utama yang menampilkan kartu-kartu *Workspace* secara *grid*. Tombol *"+ Buat Workspace Baru"* ditempatkan secara menonjol untuk memudahkan inisiasi proyek baru.



Gambar 11

4.2.3 Halaman Board Kanban

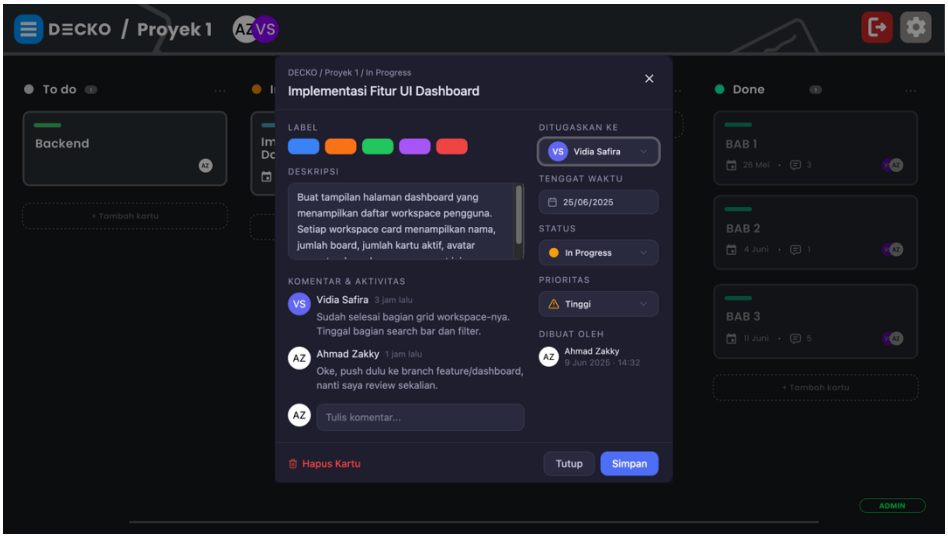
Halaman ini merupakan inti dari aplikasi Decko dan merepresentasikan implementasi metode Kanban secara visual. *Wireframe* menampilkan papan Kanban dengan tata letak horizontal yang terdiri dari kolom-kolom *List* yang dapat digulir ke samping. Setiap kolom memuat kartu-kartu tugas (*Card*) yang dapat dipindahkan antar kolom melalui mekanisme *drag-and-drop*. Header halaman menampilkan nama *Board* beserta tombol manajemen anggota tim.



Gambar 12

4.2.4 Modal Detail Kartu Tugas (Card Detail)

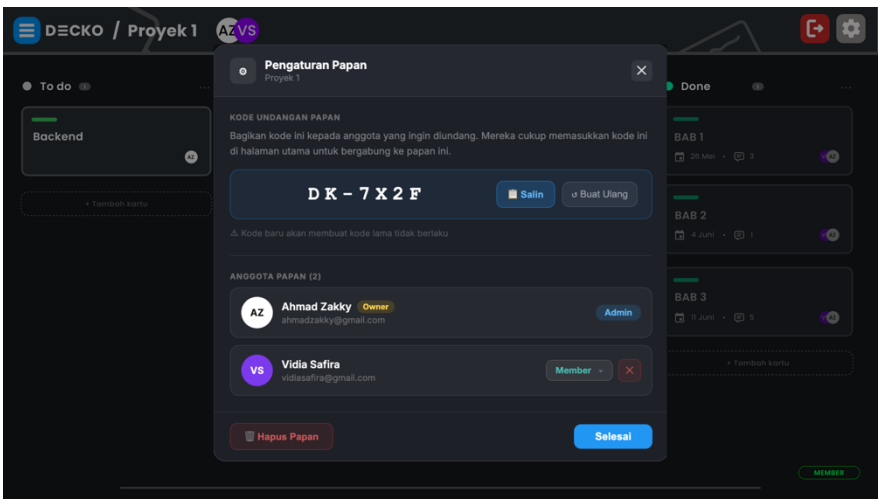
Saat pengguna mengklik sebuah kartu tugas, sistem menampilkan modal dialog yang berisi detail lengkap pekerjaan. Modal ini mencakup area judul yang dapat diedit, kolom deskripsi, informasi penugasan anggota tim, serta pilihan label warna. Desain modal menggunakan lapisan overlay agar papan Kanban di belakangnya tetap terlihat, memberikan konteks visual posisi kartu dalam alur kerja.



Gambar 13

4.2.5 Modal Pengaturan Papan dan Manajemen Anggota Tim

Halaman ini hanya dapat diakses oleh pengguna dengan peran Owner dan Admin. Tampilan menyajikan daftar seluruh anggota yang terdaftar di dalam *Workspace* beserta peran masing-masing (*Owner*, *Admin*, *Member*, atau *Observer*).Admin dapat mengubah peran anggota, menghapus anggota dari ruang kerja, serta mengundang pengguna baru melalui kode undangan. Halaman ini merefleksikan implementasi sistem RBAC yang telah dirancang pada arsitektur sistem.



Gambar 14

4.3 Deskripsi Tampilan

Bagian ini menjelaskan fungsi dan komponen dari setiap halaman utama yang dirancang pada aplikasi Decko. Penjelasan ini mencakup tujuan halaman, elemen-elemen antarmuka yang terdapat di dalamnya, serta keterkaitan halaman tersebut dengan alur kerja dan sistem RBAC yang telah ditetapkan.

Tabel 2 Deskripsi Halaman Decko

No.	Nama Halaman	Fungsi Utama	Hak Akses
1	Halaman Login	Memvalidasi kredensial pengguna berdasarkan *username* & *password* dan menghasilkan token JWT untuk sesi aktif.	Semua Pengguna
2	Halaman Register	Memproses pendaftaran akun baru dengan mengumpulkan data *email*, *username*, dan *password*, lalu menyimpan data pengguna ke dalam database dengan password terenkripsi.	Semua Pengguna
3	Dashboard (Daftar Workspace)	Menampilkan seluruh *Workspace* yang dimiliki atau diikuti pengguna berdasarkan validasi token keanggotaan, serta menyediakan akses untuk membuat *Workspace* baru.	Owner, Admin, Member, Observer
4	Halaman Board Kanban	Menampilkan papan Kanban dengan kolom *List* dan kartu *Card* secara visual. Mendukung operasi *drag-and-drop* (*batch-update* posisi via order_index), penampilan kode undangan (*invitation code*), serta akses ke *Settings Modal* manajemen	*Owner, Admin, Member* (full edit); *Observer* (view only)

No.	Nama Halaman	Fungsi Utama	Hak Akses
		anggota tim. Sinkronisasi data digerakkan secara *real-time* menggunakan Socket.io.	
5	Modal Detail Card	Menampilkan dan memungkinkan pengeditan detail kartu tugas secara mendalam, termasuk judul, deskripsi, alokasi *card_members*, warna *labels*, penentuan *priority*, pengaturan *deadline*, serta kolom interaksi *comments*.	*Owner, Admin, Member* (edit & comment); *Observer* (view only)
6	Manajemen Anggota Tim	Tampilan modal popup di dalam halaman *Board* yang menampilkan daftar anggota *Workspace*. Memungkinkan pengelolaan peran anggota tim dan pembuatan ulang kode undangan.	Owner dan Admin saja

4.3.1 Halaman Autentikasi

Halaman Autentikasi merupakan lapisan pertama keamanan aplikasi Decko. Pada halaman Login, pengguna memasukkan *username* dan *password*. Apabila berhasil, *backend* berbasis Express akan menerbitkan token JWT (*JSON Web Token*) yang disimpan di sisi *client* untuk disertakan pada setiap *request header* API sebagai mekanisme autentikasi *stateless*. Halaman Register memungkinkan pengguna baru mendaftarkan akun dengan data *email*, *username*, dan *password* yang akan disimpan di entitas users dengan kata sandi hasil *hashing*. Desain antarmuka dibuat berpusat di tengah layar (*centered form*) dengan dominasi tema gelap minimalis agar pengguna dapat fokus melakukan autentikasi dengan cepat.

4.3.2 Halaman Dashboard

Halaman Dashboard berfungsi sebagai pusat navigasi utama setelah pengguna berhasil masuk. Halaman ini membaca data dari entitas workspace dan workspace_members untuk memuat daftar seluruh ruang kerja yang relevan bagi pengguna yang sedang aktif. Tampilan menyajikan kartu-kartu *Workspace* secara *grid* dengan sidebar navigasi di sisi kiri. Pada halaman ini tersedia tombol "+ Buat Workspace Baru", di mana pengguna yang menginisiasi pembuatan otomatis terdata sebagai *Owner* di dalam basis data ruang kerja tersebut

4.3.3 Halaman Board Kanban

Halaman Board Kanban adalah inti utama dari aplikasi Decko yang merepresentasikan implementasi metode Kanban secara visual dan *real-time*. Antarmuka halaman ini menampilkan papan Kanban dengan tata letak kolom horizontal (list) yang dapat digulirkan ke samping. Setiap kolom memuat kartu-kartu tugas (card) yang diurutkan secara vertikal berdasarkan parameter order_index. Mekanisme *drag-and-drop* pada halaman ini berinteraksi langsung dengan RESTful API *batch-update* di *backend* Express dan *gateway* Socket.io. Apabila pengguna berstatus *Observer* mencoba memindahkan kartu, *middleware* RBAC akan mendeteksi pelanggaran hak akses, mengembalikan status eror, dan antarmuka React akan membatalkan posisi kartu di layar secara instan seraya menampilkan notifikasi penolakan akses. Di bagian *header* papan, terdapat tampilan *invitation code* ruang kerja serta tombol untuk memicu *Settings Modal*.

4.3.4 Modal Detail Kartu Tugas

Komponen *Modal Detail Card* menyediakan akses ke informasi lengkap dari setiap kartu tugas tanpa harus meninggalkan tampilan papan Kanban. Komponen ini berupa *popup overlay* yang terhubung ke data entitas card di PostgreSQL dan mendukung operasi CRUD. Di dalam modal ini, pengguna dengan peran *Owner*, *Admin*, atau *Member* dapat mengubah judul, deskripsi, memilih tingkatan *priority*, mengatur kalender *deadline*, menambahkan *labels*, memilih anggota tim penanggung jawab (card_members), serta menuliskan ulasan pada

kolom comments. Validasi hak akses tetap diterapkan secara ketat oleh *backend*, di mana pengguna dengan peran *Observer* hanya dapat membaca seluruh data tersebut tanpa hak akses manipulasi ataupun menuliskan komentar.

4.3.5 Settings Modal (Manajemen Anggota Tim)

Settings Modal dirancang secara eksklusif sebagai pengganti halaman terpisah untuk kebutuhan pengelolaan sistem RBAC secara praktis. Komponen ini dapat diakses oleh *Owner* dan *Admin* melalui tombol opsi di header *Board*. Di dalam modal ini ditampilkan daftar seluruh anggota tim yang terikat pada workspace_members. *Owner* dan *Admin* dapat memantau nama akun pengguna, menyunting tingkatan peran anggota (misal mengubah *Member* menjadi *Admin* atau menurunkan menjadi *Observer*), serta memutuskan hak keanggotaan. *Owner* juga memiliki opsi eksklusif di dalam modal ini untuk menyegarkan atau membuat ulang *invitation code* baru guna mengamankan akses masuk ruang kerja dari pihak luar. Perubahan peran pada modal ini akan langsung mempengaruhi otorisasi akses pengguna secara *real-time* di seluruh papan proyek dalam ruang kerja tersebut.